TUNIS BUSINESS SCHOOL
UNIVERSITY OF TUNIS

SUBJECT:
Information Assurance and Security

PROJECT:
Market Assessment of Electric Vehicles using Web Scraping

PROFESSOR:
Mrs.Manel Abdelkader

INSTITUTION:
Tunis Business School

AUTHORS:
Eya Maalej - Kmar Abessi - Meriem Houissa - Salma Azouzi - Sarra Jebali

MAJOR BA, MINOR IT
ACADEMIC YEAR: 2024-2025

TABLE OF CONTENTS

INTRODUCTION

IMPLEMENTATION

1.Car Listings

- 1.1 Scraping
- 1.2 Data Cleaning
- 1.3 Dashboarding
- 1.4 Web-Page Integration

2. Sentiment Analysis

- 2.1 Scraping
- 2.2 Cleaning
- 2.3 Analysis

3. Interpretations

CHALLENGES

CONCLUSION

INTRODUCTION

Context & Problematic

- **Market Challenge:** Electric vehicle (EV) data is fragmented across multiple sources
- **User Pain Points:**
  - Information scattered across dynamic websites
  - Inconsistent data formats
  - Difficult to make direct comparisons
  - Time-consuming to gather comprehensive market insights

INTRODUCTION

Objectives

- Create a single source of truth for electric vehicle data
- Simplify EV research and comparison for consumers
- Enhance decision-making for potential EV buyers

Final solution

- Collected data through scraping 4 websites & cleaning
- Built dashboards for each website for specific filtering
- Created a web application with cross-source filtering
- Conducted sentiment analysis from TrustPilot using NLP

IMPLEMENTATION

1.Car Listings

Websites Scraped:

Technology Stack:

Editors

Language

IMPLEMENTATION

Liabraries

Library	Purpose
selenium	Automates browser actions to interact with dynamic web pages and extract data
selenium.webdriver.chrome.options.Options	Configures Chrome browser options like headless mode, disabling sandbox, and resource usage
selenium.webdriver.common.by.By	Provides methods to locate web elements by CSS selectors, XPath, class names, etc.
bs4 (BeautifulSoup)	Parses static HTML content into a structured format for easier data extraction
time	Introduces delays to allow pages to load fully before scraping
pandas	Structures scraped data into DataFrames and exports data to CSV files
sys	Modifies system path to enable ChromeDriver access in certain environments (e.g., Colab)
google.colab.files	Facilitates file downloading within Google Colab notebooks

IMPLEMENTATION

1.1 Scraping

Unified Approach (with site-specific tweaks):

1. Setup: Selenium WebDriver (headless) for dynamic content
2. Pagination: Loop through all pages to gather listings
3. Data Extraction: Use CSS selectors/XPath for key info
4. Detail Pages: Visit if more specs needed
5. Storage: pandas DataFrame → CSV

➡ **Logic: Setup → Get Listings → Extract Details → Save**

NEXT SLIDE: example EV.com

IMPLEMENTATION

1.1 Scraping

```
from selenium import webdriver
from selenium.webdriver.chrome.options import Options
from selenium.webdriver.chrome.service import Service
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from bs4 import BeautifulSoup
import time
import random
import json
import re
from urllib.parse import urljoin
import pandas as pd
import logging
import os
```

importing necessary libraries :

- Selenium
- BeautifulSoup
- time
- random
- json
- re
- urllib.parse
- pandas
- logging
- os

IMPLEMENTATION

1.1 Scraping

```python
def setup_driver():
    """Configure Chrome WebDriver with optimized settings for headless browsing"""
    options = Options()
    options.add_argument("--headless")
    options.add_argument("--disable-dev-shm-usage")
    options.add_argument("--no-sandbox")
    options.add_argument("--window-size=1920,1080")
    options.add_argument("--disable-blink-features=AutomationControlled")
    options.add_argument("user-agent=Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, l
    options.add_experimental_option("excludeSwitches", ["enable-automation"])
    options.add_experimental_option("useAutomationExtension", False)

    # Specify the correct path to the ChromeDriver executable
    chromedriver_path = r"C:\Users\EyaMaalej\Desktop\chromedriver-win64\chromedriver.exe"
    service = Service(executable_path=chromedriver_path)
    driver = webdriver.Chrome(service=service, options=options)

    driver.set_page_load_timeout(90)
    logging.info(f"ChromeDriver initialized with Chrome version: {driver.capabilities['browserVersion']}")
    return driver
```

- Sets up **a headless Chrome browser**
- Hides automation traces
- Loads ChromeDriver from a given path
- Sets **90s page load timeout**
- Returns the driver

IMPLEMENTATION

1.1 Scraping

```
def extract_vehicle_details(detail_url, idx, brand):
```

This function extracts **detailed specifications** for a specific vehicle from its detail page using **Selenium, BeautifulSoup, RegEx, and error handling.**

Process:

- Navigates to the vehicle's detail URL
- Waits for the page to fully load
- Extracts key specifications from the grid and list sections
- Captures title and price information
- Implements retry logic for reliability
- Saves page source for debugging purposes

IMPLEMENTATION

1.1 Scraping

- Tries 3 times to load a vehicle detail page
- Waits for full page load
- Waits for key elements to appear
- Adds a random delay to avoid detection
- Ensures reliable data extraction

```
max_attempts = 3
for attempt in range(max_attempts):
    try:
        print(f"Extracting vehicle {idx} for {brand}: {detail_url} (Attempt {attempt + 1}/{max_attempts})")
        logging.info(f"Starting extraction for vehicle {idx} of {brand}: {detail_url} (Attempt {attempt + 1}/{max_attempts})")
        driver.get(detail_url)

        WebDriverWait(driver, 30).until(
            lambda d: d.execute_script("return document.readyState") == "complete"
        )
        WebDriverWait(driver, 20).until(
            EC.presence_of_element_located(
                (By.XPATH, "//div[contains(@class, 'grid') or contains(@class, 'spec') or contains(@class, 'text-body1Light')]")
            )
        )
        time.sleep(random.uniform(3, 5))
```

IMPLEMENTATION

1.1 Scraping

These are some code snippets from the extract_vehicle_details Function

```
# Extract grid section details
grid_sections = soup.find_all("div", class_=lambda x: x and 'grid' in x)
for grid in grid_sections:
    items = grid.find_all("div", recursive=False)
    for item in items:
        text_lines = [line.strip() for line in item.get_text("\n").split("\n") if line.strip()]
        if len(text_lines) >= 2:
            spec_name = text_lines[0]
            spec_value = "\n".join(text_lines[1:])
            if any(x in spec_name.lower() for x in ['range', 'distance']):
                details['Range'] = spec_value
            elif 'fast charging' in spec_name.lower():
                details['Fast Charging L3'] = spec_value
            elif 'mileage' in spec_name.lower():
                details['Mileage'] = spec_value
            elif 'performance' in spec_name.lower():
                details['Performance'] = spec_value
            elif 'seats' in spec_name.lower():
                details['Seats'] = spec_value
            elif 'battery size' in spec_name.lower():
                details['Battery size'] = spec_value
```

```
# Extract title
details['Title'] = soup.find("h1").get_text(strip=True) if soup.find("h1") else "N/A"
```

```
# Extract list section details
list_items = soup.find_all("div", class_=lambda x: x and 'text-body1Light' in x)
for item in list_items:
    spec_name = item.get_text(strip=True)
    value_div = item.find_next("div", class_=lambda x: x and 'text-right' in x)
    if value_div:
        spec_value = value_div.get_text(strip=True)
        if 'exterior color' in spec_name.lower():
            details['Exterior color'] = spec_value
        elif 'interior color' in spec_name.lower():
            details['Interior color'] = spec_value
        elif 'charging type' in spec_name.lower():
            details['Charging type'] = spec_value
        elif 'battery warranty' in spec_name.lower():
            details['Battery warranty'] = spec_value
        elif 'vin' in spec_name.lower():
            details['VIN'] = spec_value
        elif 'year' in spec_name.lower():
            details['Year'] = spec_value
```

IMPLEMENTATION

1.1 Scraping

```
def process_listing_page(listings_url, brand, page_num, vehicle_idx, processed_urls):
```

This function processes a listing page to identify and extract information for individual vehicles using the extract_vehicle_details function already defined.

Process:

- Navigates to the listing page URL
- Waits for the page to load fully
- Identifies listing elements
- Extracts vehicle detail URLs
- Calls extract_vehicle_details for each vehicle
- Tracks already processed URLs to avoid duplicates
- Returns collected data and updated vehicle index

IMPLEMENTATION

1.1 Scraping

```python
def generate_page_url(base_url, brand, page_num):
    """Generate the URL for a specific page of a brand"""
    if page_num == 1:
        return f"{base_url}/make/{brand}"
    else:
        return f"{base_url}/distance/200/make/{brand}/sort/rankedscore_desc/has_ev_incentives/false/incentive_zip/90013/page/{page_num}"
```

This function generates URLs for different pages of brand-specific vehicle listings using string formatting

Process:

- Creates different URL formats for the first page vs. subsequent pages
- Includes parameters for distance, sorting and incentives

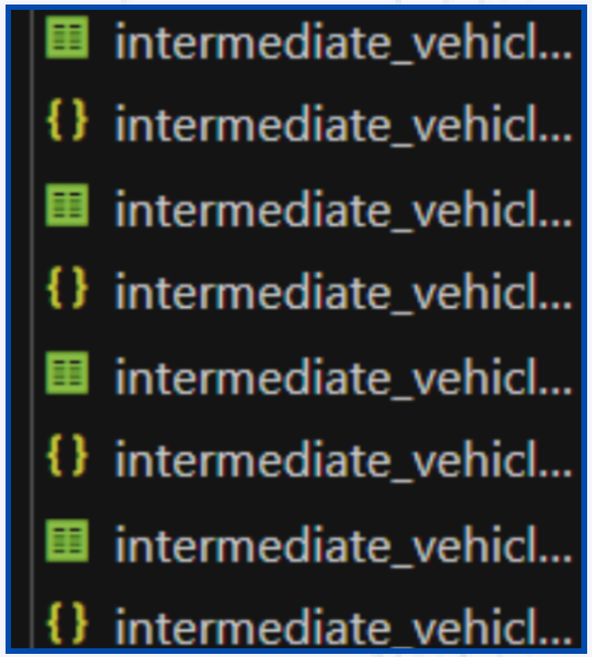
This step was done after inspecting the URLs structure from the website

IMPLEMENTATION

1.1 Scraping

```
def save_intermediate_results(all_vehicle_data, brand):
```

This function saves intermediate results after processing each brand using Pandas, JSON, os module



CSV file with all vehicle data
JSON file with all vehicle data
Saved in the “output” directory

```
df = pd.DataFrame(all_vehicle_data)
output_csv = os.path.join(OUTPUT_DIR, f'intermediate_vehicle_data_{brand}.csv')
output_json = os.path.join(OUTPUT_DIR, f'intermediate_vehicle_data_{brand}.json')
df.to_csv(output_csv, index=False, encoding='utf-8-sig')
with open(output_json, 'w', encoding='utf-8') as f:
    json.dump(all_vehicle_data, f, indent=4)
print(f"Saved intermediate results for {brand} to {output_csv} and {output_json}")
logging.info(f"Saved intermediate results for {brand} to {output_csv} and {output_json}")
```

IMPLEMENTATION

1.1 Scraping

```
if __name__ == "__main__":
    # Base search URL
    base_search_url = "https://ev.com/search/postal/90013"

    # List of brands to scrape
    brands = ['Tesla', 'Chevrolet', 'Kia', 'Ford', 'Hyundai', 'Rivian', 'Acura', 'Alfa Romeo', 'Audi', 'BMW',
              'Bentley', 'Cadillac', 'Chrysler', 'Dodge', 'FIAT', 'GMC', 'Genesis', 'Honda', 'Jeep',
              'Lamborghini', 'Land Rover', 'Lexus', 'Lincoln', 'Lucid', 'MINI', 'Maserati', 'Mazda',
              'Mercedes-Benz', 'Nissan', 'Polestar', 'Porsche', 'Subaru', 'Toyota', 'VinFast',
              'Volkswagen', 'Volvo']

    # Initialize data storage
    all_vehicle_data = []
    processed_urls = set()
    max_pages = 30

    print(f"Starting scraping for {len(brands)} brands: {', '.join(brands)}")
    logging.info(f"Starting scraping process for {len(brands)} brands")
```

```
# Process each brand
for brand in brands:
```

```
for page_num in range(1, max_pages + 1):
    page_url = generate_page_url(base_search_url, brand, page_num)
    print(f"Fetching {brand} page {page_num}: {page_url}")
    logging.info(f"Generated URL for {brand} page {page_num}")
```

This part orchestrates the entire scraping process using Python control structures and logging.

Process:

- Defines base search URL and list of brands to scrape
- Creates output directory if it doesn't exist

For each brand:

- Processes each page of results (up to 30 pages)
- Collects vehicle data
- Saves intermediate results
- Combines all data into final CSV and JSON files
- Logs completion statistics

IMPLEMENTATION

1.1 Scraping

ERROR handling:

- Multiple retry attempts for failed operations
- Detailed logging for debugging
- Screenshot capture of errors
- Saving of page source for failed pages

Logging:

The script uses Python's logging module to track execution:

- Logs are formatted with timestamps and severity levels
- Captures INFO, WARNING, and ERROR messages
- Useful for debugging and tracking progress

IMPLEMENTATION

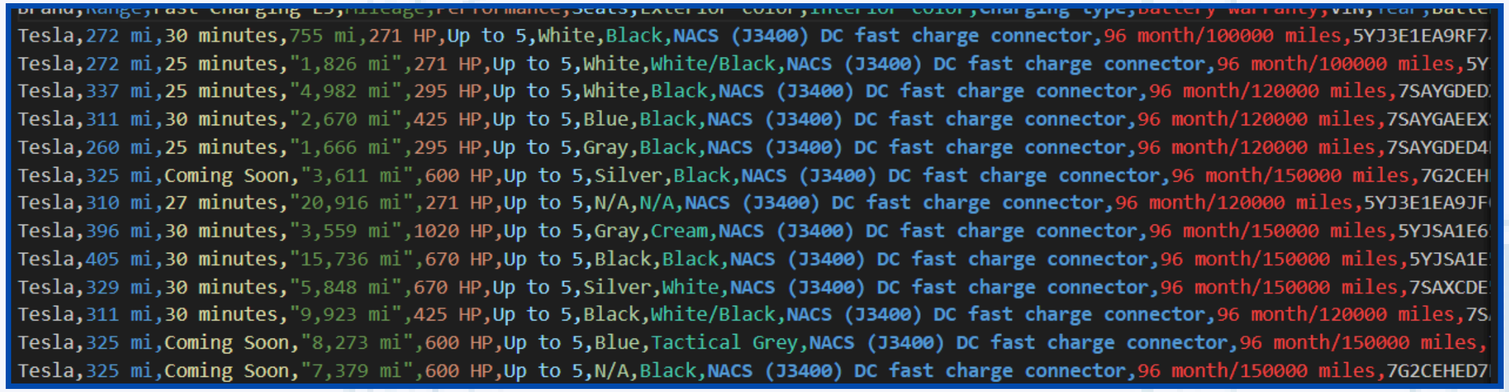
1.1 Scraping

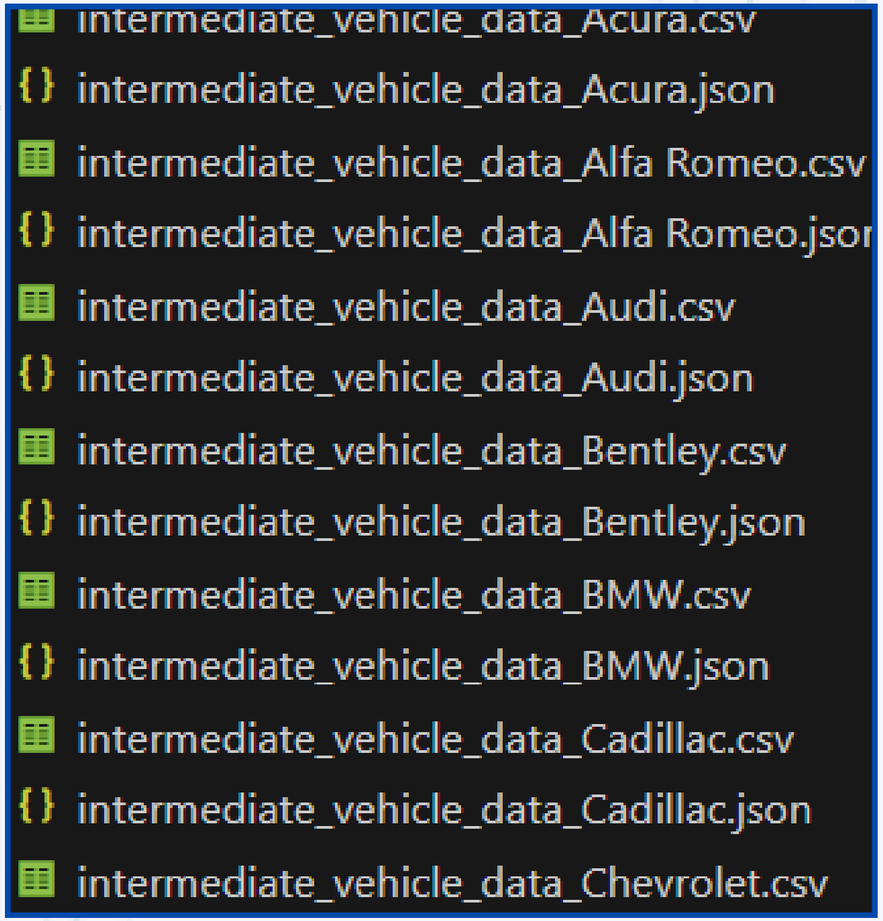
OUTPUT

CSV & JSON

Intermediate

Final





IMPLEMENTATION

1.1 Scraping

Differences & Reasons

ELECTRIFYING.com

Simpler approach with a single driver, scraping mostly from listing pages and some details from individual pages.

More complex, using separate drivers for listings and detail pages since detailed specs require extra navigation and session handling.

EV.com

Uses one driver but visits each detail page to balance data completeness with efficiency on dynamic content.

One driver combined with BeautifulSoup scrapes only listing cards, adapting to site structure.

IMPLEMENTATION

1.2 Data Cleaning

This part outlines the common data preprocessing workflow applied across four datasets.
Despite slight differences in structure and rawness, the core data cleaning strategy followed a standardized strategy to transform inconsistent and semi-structured data into clean, reliable, and analysis-ready datasets.

Overall process

- Initial Setup and Data Loading
- Cleaning and Formatting Numeric Columns
- Standardizing and Engineering Categorical Features
- Feature Engineering and Classification
- Filtering, Deduplication, and Final Cleaning
- Exporting Cleaned Dataset

IMPLEMENTATION

1.2 Data Cleaning

Cleaning Final Objectives

Objective	Description
✅ Data Consistency	Unify formats across numeric and categorical fields for meaningful comparison.
✅ Analysis Readiness	Prepare data for plotting trends, distributions, or building dashboards.
✅ Insight Extraction	Enable classification by price or efficiency, comparison by brand or age.
✅ Feature Engineering	Add derived variables that provide added analytical value (e.g., price per battery kWh).
✅ Clustering and Segmentation	Lay groundwork for categorizing vehicles into tiers or market segments.

IMPLEMENTATION

1.2 Data Cleaning

Initial Set-Up and Data Loading

Load data from .csv files into pandas DataFrames for cleaning.

Actions:

- Import required libraries (pandas, numpy, re).
- Upload dataset using Google Colab's files.upload().
- Read the .csv file using pd.read_csv().

```
# STEP 1:Import libraries
import pandas as pd
import numpy as np
import re
from google.colab import files


# STEP 2: Upload the CSV file
uploaded = files.upload()
# Load the data
df = pd.read_csv('electric_cars_1.csv')
```

IMPLEMENTATION

1.2 Data Cleaning

Cleaning and Formatting Numeric Columns

Standardize key numeric attributes (e.g., price, battery size, range, mileage) by removing units, symbols, and converting to numerical types.

```
# List of columns to convert
numeric_cols = [
    'Range (mi)',
    'Fast Charging L3 (minutes)',
    'Mileage (mi)',
    'Performance (HP)',
    'Seats',
    'Year',
    'Battery Size (kWh)',
    'Price ($)',
    'Battery Warranty (months)',
    'Battery Warranty (miles)'
]

# Remove any non-numeric characters (like $, commas, etc.) and convert
for col in numeric_cols:
    df[col] = pd.to_numeric(df[col].replace('[^\d.]', '', regex=True), errors='coerce')
```

Remove currency symbols (£, $) and thousands separators (,) from price.

IMPLEMENTATION

1.2 Data Cleaning

Cleaning and Formatting Numeric Columns

```
# Clean columns with units
for col, (unit, preprocess) in columns_with_units.items():
    if col in df.columns:
        # Store original values
        original_values = df[col].copy()
        # Apply preprocessing (e.g., remove commas for Price)
        df[col] = df[col].apply(lambda x: preprocess(x) if pd.notna(x) else x)
        # Remove unit and keep as string
        df[col] = df[col].astype(str).str.replace(unit, '', regex=False).str.strip()
        # Preserve "N\A" and handle failed unit removal
        mask = df[col].str.contains(unit, regex=False, na=False)
        if mask.any():
            print(f"Warning: Some values in '{col}' still contain '{unit}' after cleaning:")
            print(df.loc[mask, col].value_counts().head())
            # Revert to original values where unit removal failed
            df.loc[mask, col] = original_values[mask]
        # Ensure "N\A" is preserved
        df[col] = df[col].where(~df[col].str.lower().eq('nan'), original_values)
```

- **Strip units like kWh, mi, km from columns such as:**
  - battery_size
  - range
  - miles_per_kwh / mileage

IMPLEMENTATION

1.2 Data Cleaning

Cleaning and Formatting Numeric Columns

```
# Calculate average range and fill missing values
avg_range = round(df['range'].mean())
df['range'] = df['range'].fillna(avg_range).astype(int)
df = df.drop(columns=['range '])  # Remove the original column with space
```

- Handle ranges (e.g., "40-60") by converting them to mean values.

```
# 7. Performance (HP): Impute with median by Model, Brand
df1['Performance (HP)'] = df1.groupby(['Model', 'Brand'])['Performance (HP)'].transform(lambda x: x.fillna(x.median()))
df1['Performance (HP)'] = df1['Performance (HP)'].fillna(df1['Performance (HP)'].median())

# 8. Battery Size (kWh): Impute with median by Model, Brand, Year
df1['Battery Size (kWh)'] = df1.groupby(['Model', 'Brand', 'Year'])['Battery Size (kWh)'].transform(lambda x: x.fillna(x.median()))
df1['Battery Size (kWh)'] = df1['Battery Size (kWh)'].fillna(df1['Battery Size (kWh)'].median())
```

- Replace missing or malformed entries with Column mean or median for example

IMPLEMENTATION

1.2 Data Cleaning

Standardizing and Engineering Categorical Features

Extract or clean brand, model, and year data to allow grouping and comparative analysis.

```
def extract_make_model(title):
    parts = title.split()
    year = parts[0]
    rest = " ".join(parts[1:])

    for make in compound_makes:
        if rest.startswith(make):
            return pd.Series([year, make, rest[len(make):].strip()])

    # Fallback to one-word make
    return pd.Series([year, parts[1], " ".join(parts[2:])])

# Apply extraction
df_clean[['Year', 'Make', 'Model']] = df_clean['Title'].apply(extract_make_model)
```

Split compound name columns to isolate the make and model.

IMPLEMENTATION

1.2 Data Cleaning

Feature Engineering and Classification

Derive insightful new variables to enable further analysis, clustering, or modeling.

```
# Create the Battery Efficiency column
df["Battery Efficiency"] = df["Battery (kWh)"] / df["Engine Capacity (p.h.)"]
df["Price per kWh"] = df["Price ($)"] / df["Battery (kWh)"]
df["Performance Index"] = (df["Battery (kWh)"] * df["Engine Capacity (p.h.)"]) / df["Price ($)"]
df["Car Age"] = 2025 - df["First Registration"]
df["Engine Category"] = pd.cut(df["Engine Capacity (p.h.)"],
                               bins=[0, 150, 300, 600, 1200],
                               labels=["Small", "Medium", "High", "Extreme"])
```

```
# Reorder cluster labels based on actual price
cluster_centers = kmeans.cluster_centers_.flatten()
sorted_indices = np.argsort(cluster_centers)
cluster_to_label = {sorted_indices[0]: 'Cheap',
                    sorted_indices[1]: 'Midrange',
                    sorted_indices[2]: 'Luxury'}
```

efficiency_score: Based on mileage or range per battery unit.

price_category: Classify vehicles into "Cheap", "Midrange", "Luxury".

IMPLEMENTATION

1.2 Data Cleaning

Filtering, Deduplication, and Final Cleaning

Remove irrelevant or duplicated entries and finalize dataset integrity.

```
# STEP 4: Remove duplicates
df = df.drop_duplicates()
```

Drop duplicates using df.drop_duplicates().

```
# Drop rows where 'VIN' is missing
df1 = df1.dropna(subset=['VIN'])
```

Filter out rows with critical missing values.

IMPLEMENTATION

1.2 Data Cleaning

Exporting Cleaned Dataset

Save the cleaned data for further analysis, visualization, or modeling.

```
# Save cleaned data
df.to_csv('electric_cars_cleaned.csv', index=False)
```

Export final dataset using df.to_csv()

```
files.download('electric_cars_cleaned.csv')
```

Use files.download() in Colab to download the output file.

IMPLEMENTATION

1.2 Data Cleaning

Tools & Libraries

Aspect	How It's Handled Across All Sites
Unit and symbols Removal	.replace() and regex to remove units like kWh, km/h, €, £
Numeric Conversion	int() or float() after cleaning strings
Missing Data Handling	Use np.nan for missing/invalid data
Stripping Whitespace	.strip() applied to clean trailing spaces
Regex Use	Used in all scripts for extracting numeric values

IMPLEMENTATION
1.3 Dashboarding

This dashboard visualizes key metrics such as the **number of car brands**, **dealer activity by location**, **pricing trends over time**, **vehicle models, and value scores.** The dashboard also categorizes vehicles by **price class** and analyzes **the relationship between year, mileage, and price.**

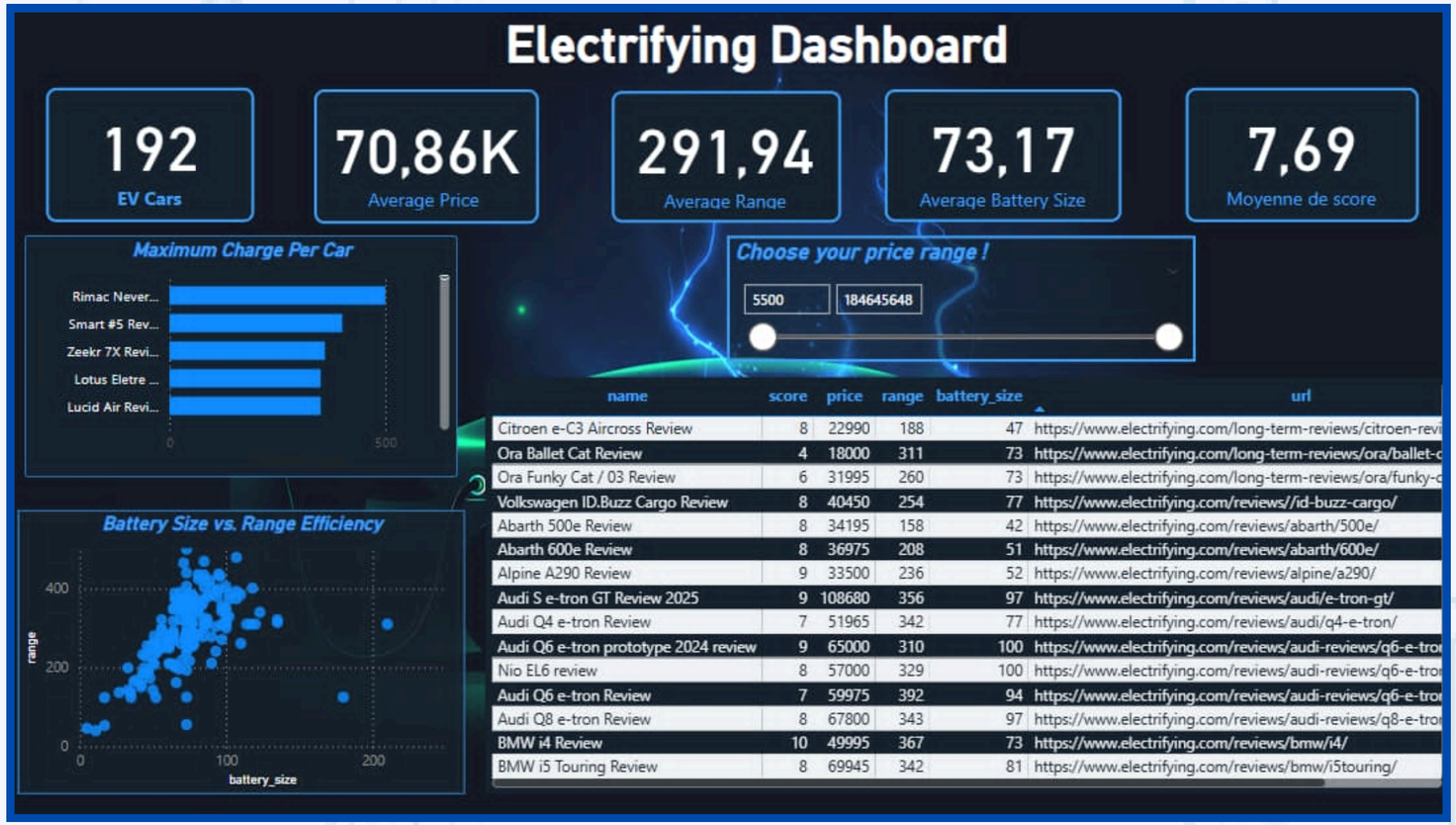
Objectives

- Monitor market trends and dealer activity in the electric vehicle sector
- Identify pricing patterns and detect anomalies or suspicious listings
- Assess brand performance and vehicle value scores
- Provide actionable insights for timely decision-making and risk mitigation

IMPLEMENTATION
1.3 Dashboarding

ELECTRIFYING.COM

This dashboard visualizes **driving range**, **battery size, and user ratings.** It combines summary statistics, interactive filters, and detailed data tables to provide a **clear overview of vehicle performance and pricing trends.** Visual elements like the battery charge bar chart and battery size versus range scatter plot help identify efficiency patterns and outliers, supporting market analysis and informed decision-making.

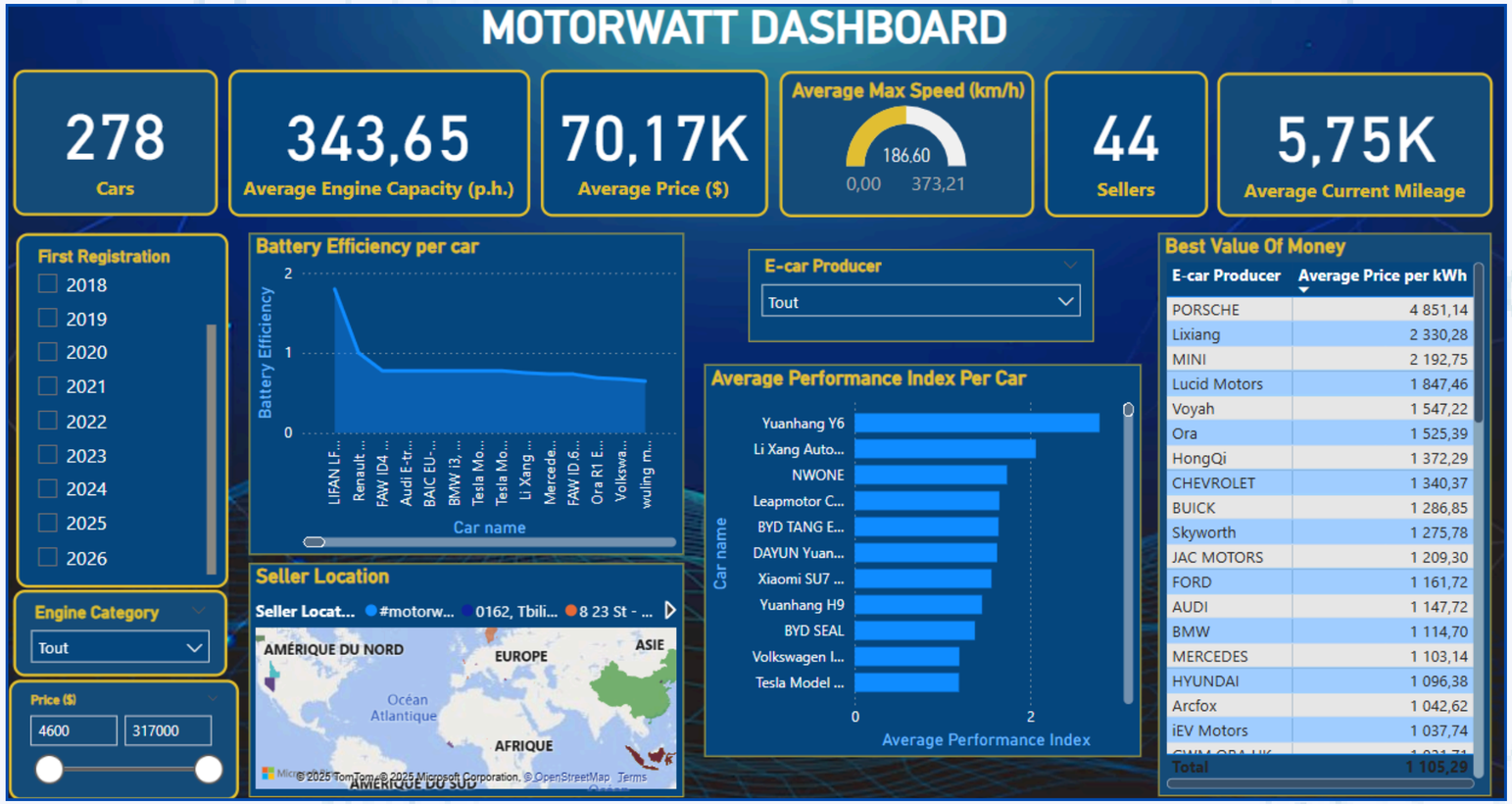


Objectives

- Compare driving range and battery capacity to assess efficiency and technological advancements across models.
- Detect unusual values in price, battery size, or range that could indicate errors, misreporting, or potential fraud.
- Offer easy access to detailed reviews and ratings to aid buyers in vehicle selection.
- Allow users to filter vehicles by price, enhancing the dashboard's usability for targeted analysis.

IMPLEMENTATION
1.3 Dashboarding

This dashboard features visual analyses of **battery efficiency per car, seller locations on a geographic map, and the average performance index of different car models.** Additionally, it highlights the **best value-for-money rankings by car producers** and provides detailed filtering options such as first registration year, engine category, and price range to facilitate in-depth market exploration.

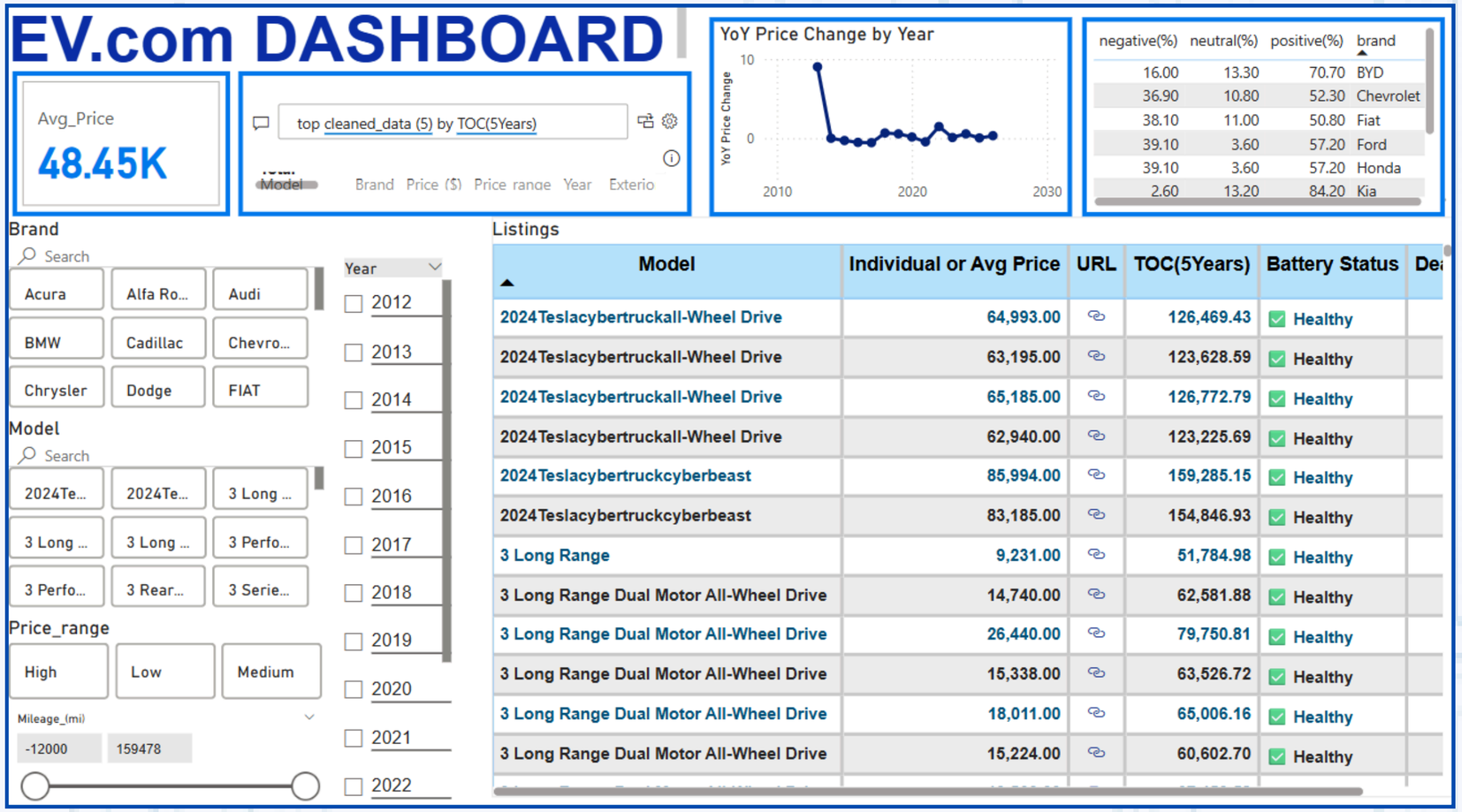


Objectives

- Map seller locations to understand the geographic distribution and density of dealerships.
- Present value-for-money comparisons by car producer to guide consumers and highlight competitive pricing.
- Enable market segmentation and detailed filtering for targeted analysis of vehicle data by year, engine type, and price

IMPLEMENTATION
1.3 Dashboarding

The Dashboard visualizes key metrics such as **average vehicle price, year-over-year price changes, and brand sentiment analysis.** Users can filter data by car brand, model, year, price range, and mileage, enabling detailed exploration of listings. The table shows specific **model listings, prices, battery health, and total ownership costs over five years, offering a detailed market snapshot**.



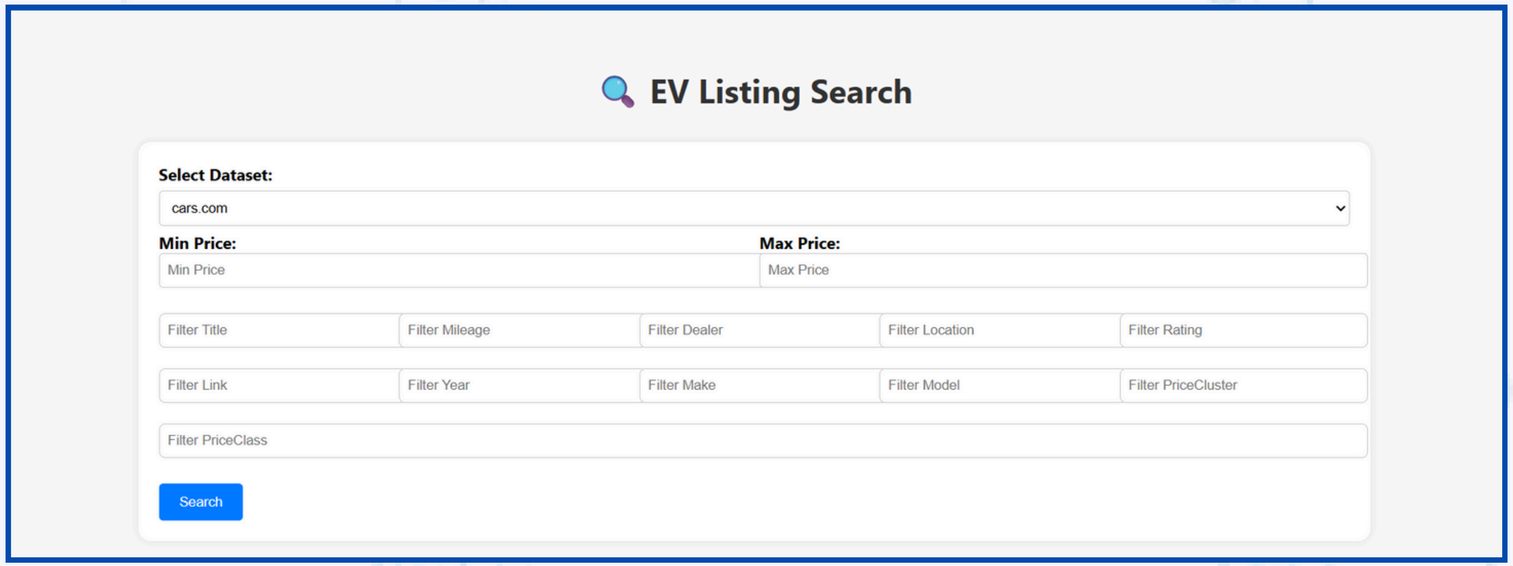
Objectives

- Assess customer sentiment by brand, highlighting positive, neutral, and negative feedback percentages.
- Enable dynamic filtering by brand, model, year, price range, and mileage for targeted insights.
- Provide detailed listings with pricing, battery health status, and total cost of ownership to support buyer decision-making.

IMPLEMENTATION

1.4 Web-Page Integration

goal : Unify the collected data from 4 EV websites in a single interface



solution **:** Created a **lightweight web app using Flask**

→ Allows filtering, exploring, , comparing EV listings, and informative powerbi dashboards visualization

IMPLEMENTATION

1.4 Web-Page Integration

TECHNOLOGY STACK

Tool	Purpose
Flask : A Python micro-framework to quickly build web apps	Serve pages and handle backend logic
Pandas	Load, filter, and manage CSV data
jinja2 : templating engine for Python—used to generate dynamic HTML in web apps built. Embeds Python-like logic directly into your HTML files using special syntax.	Loop through datasets, Fill in form inputs with previous filter values, Display a dynamic table of car listings, Show or hide a “View Listing” button only when there's a link
HTML/CSS	Build clean and responsive UI

IMPLEMENTATION

1.4 Web-Page Integration

1 *Multi-source Dataset Selection*

Users can choose among four major EV data sources: cars.com, EV.com, wattnow, and electrifying.

2 *Unified Filtering System:*

The app supports: Price range filters (min and max), brand/Make filters, flexible keyword search across any column.

3 *Column Normalization:*

Since each dataset uses different column names for price (e.g., startingPrice, price_in_usd, etc.), the app automatically identifies and renames the first price-related column to a common price field for consistent filtering.

4 Modular & Extendable Architecture:

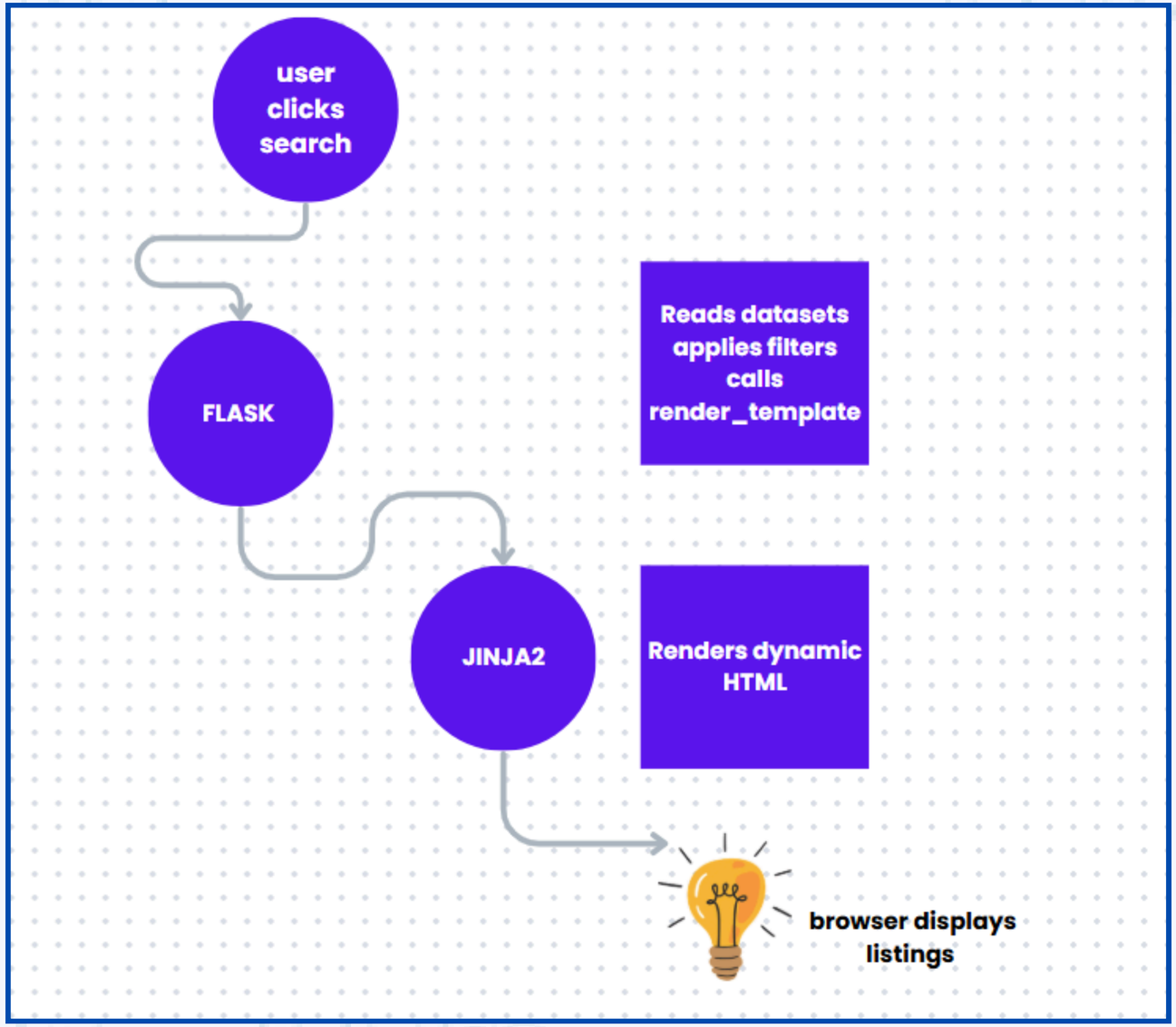
The codebase is structured to support adding more data sources, new filters, or richer comparison tools in future versions.

IMPLEMENTATION

1.4 Web-Page Integration

1. User clicks search
2. Flask reads data, applies filters, and calls render_template()
3. Jinja2 renders dynamic HTML
4. The browser displays the filtered listings

FUNCTIONAL DIAGRAM



IMPLEMENTATION
1.4 Web-Page Integration

EXECUTION STEPS

- **Flask App Setup:** Imported required libraries: Flask, pandas, os, etc.
- Initialized Flask application in app.py, defined the “/” route to serve the main page
- **Dynamic Dataset Loading:** Mapped dataset names to CSV paths, used dropdown selection in the form to switch between data sources, loaded selected dataset dynamically using pandas.read_csv()

```python
from flask import Flask, render_template, request
import pandas as pd
import os

app = Flask(__name__)

# Mapping dataset names to their paths
DATASETS = {
    'cars.com': 'datasets/classified_cars.csv',
    'EV.com': 'datasets/cleaned_data (5).csv',
    'wattnow': 'datasets/another_site.csv',
    'electrifying': 'datasets/last.csv'
}

@app.route('/', methods=['GET', 'POST'])
def index():
    selected = request.form.get('dataset') or 'cars.com'
    df = pd.read_csv(DATASETS[selected])
```

IMPLEMENTATION

1.4 Web-Page Integration

Filtering Logic

- Implemented form-based filters using request.form
- Normalized price columns across different sources
- Applied filters like:Min/Max price (converted to float), Make/BrandGeneral column text search, Ensured filters are optional and robust to bad input

```
filters = {}
filtered_df = df.copy()

if request.method == 'POST':

    min_price = request.form.get('min_price')
    max_price = request.form.get('max_price')

    if min_price and min_price.strip():
        try:
            min_price_float = float(min_price)
            filtered_df = filtered_df[filtered_df['price'].astype(float) >= min_price_float]
            filters['min_price'] = min_price
        except (ValueError, TypeError):
            pass

    if max_price and max_price.strip():
        try:
            max_price_float = float(max_price)
            filtered_df = filtered_df[filtered_df['price'].astype(float) <= max_price_float]
            filters['max_price'] = max_price
        except (ValueError, TypeError):
            pass  # Invalid input, ignore filter
```

```
    # Handle Make filter separately with exact matching
    make_filter = request.form.get('Make')
    if make_filter and make_filter.strip():
        filtered_df = filtered_df[filtered_df['Make'].astype(str).str.contains(make_filter, case=False, na=False)]
        filters['Make'] = make_filter

    # Apply other filters from the form
    for col in filtered_df.columns:
        if col not in ['price', 'Make']:  # Skip price and Make as we handle them separately
            val = request.form.get(col)
            if val and val.strip():
                filters[col] = val
                filtered_df = filtered_df[filtered_df[col].astype(str).str.contains(val, case=False, na=False)]
```

IMPLEMENTATION

1.4 Web-Page Integration

Front-End Integration (Jinja2 Template)

Created index.html template to: Display dropdown for source selection, Render filter inputs dynamically, Display filtered results

```
return render_template('index.html',
                       datasets=DATASETS.keys(),
                       selected=selected,
                       columns=filtered_df.columns,
                       filters=filters,
                       data=filtered_df.head(50).to_dict(orient='records'))
```

IMPLEMENTATION

2. Sentiment Analysis

goal : The purpose of this step is to collect user reviews for electric vehicles from Trustpilot, a popular online review platform.

This raw review data serves as the foundation for subsequent cleaning and sentiment analysis to understand customer opinions and experiences about different EV models.

IMPLEMENTATION

2. Sentiment Analysis

2.1 Scraping :

```python
# Base URL for BYD reviews
base_url = "https://www.trustpilot.com/review/byd.com"
reviews = []

# Function to extract reviews from a page
def scrape_reviews():
    time.sleep(2)  # Let the page fully load
    soup = BeautifulSoup(driver.page_source, 'html.parser')
    review_elements = soup.find_all('p', class_='typography_body-l__v5JLj typography_appearance-default__t8iAq')
    for review in review_elements:
        reviews.append(review.get_text(strip=True))

# Loop through 3 pages
for page_num in range(1, 4):
    url = f"{base_url}?page={page_num}"
    driver.get(url)
    scrape_reviews()
    print(f"✅ Scraped page {page_num}")

driver.quit()
# Print results
for idx, review in enumerate(reviews, start=1):
    print(f"\nReview {idx}:\n{review}")

# Save to CSV
df = pd.DataFrame(reviews, columns=["Review"])
csv_filename = "byd_reviews_trustpilot.csv"
df.to_csv(csv_filename, index=False)
```

- **Waits 2 seconds after each page load to ensure full content rendering**
- **Extracts<p> tags with review-text class from Trustpilot**
- **Loops through paginated URLs (pages 1 to n)**
- **Stors reviews in a Python list for analysis**

IMPLEMENTATION

2. Sentiment Analysis

2.2 Cleaning:

Case Normalization

Example:"Great" → "great"

```
]  df['clean_review'] = df['Review'].str.lower()
   # View the cleaned column
   print(df[['Review', 'clean_review']].head())
```

Trim Whitespace

```
[ ]  # Remove extra spaces
     df['clean_review'] = df['clean_review'].apply(lambda x: re.sub(r'\s+', ' ', x).strip())
     print(df[['Review', 'clean_review']].head())
```

Remove emojis

```
]  # Remove emojis
   df['clean_review'] = df['clean_review'].apply(lambda x: re.sub(r'[^\x00-\x7F]+', '', x))
   print(df[['Review', 'clean_review']].head())
```

Remove Special Characters & Numbers

Example: "Tesla3 is awesome!" → "tesla is awesome".

```
[ ]  import re

     # Remove special characters, numbers, and punctuation
     df['clean_review'] = df['clean_review'].apply(lambda x: re.sub(r'[^a-z\s]', '', x))
```

Remove Stopwords

Example: "this car is fast" → "car fast".

```
from nltk.tokenize import TreebankWordTokenizer
from nltk.corpus import stopwords
import nltk

# Download resources (if not already done)
nltk.download('stopwords')

# Setup
stop_words = set(stopwords.words('english'))
tokenizer = TreebankWordTokenizer()

# Function to remove stopwords
def remove_stopwords(text):
    tokens = tokenizer.tokenize(text)
    return ' '.join([word for word in tokens if word.lower() not in stop_words])

# Apply to DataFrame
df['clean_review'] = df['clean_review'].apply(remove_stopwords)
print(df[['Review', 'clean_review']].head())
```

IMPLEMENTATION

2. Sentiment Analysis

2.2 Cleaning :

Lemmatization

Reduces words to base forms (e.g., "running" → "run").

```
[ ] from nltk.stem import WordNetLemmatizer
    import re

    # Initialize Lemmatizer
    lemmatizer = WordNetLemmatizer()

    # Function to remove unwanted characters and perform simple tokenization
    def simple_tokenize_and_lemmatize(text):
        # Use regular expression to remove non-alphabetic characters (only keep wo
        text = re.sub(r'[^a-zA-Z\s]', '', text)

        # Tokenize by splitting on whitespace
        words = text.split()

        # Lemmatize words
        lemmatized_words = [lemmatizer.lemmatize(word) for word in words]

        # Return lemmatized text as a string
        return ' '.join(lemmatized_words)

    # Apply the function to the reviews column
    df['clean_review'] = df['clean_review'].apply(simple_tokenize_and_lemmatize)

    # Check the result
    print(df.head())
```

Correct spelling

```
from spellchecker import SpellChecker

spell = SpellChecker()

# Function to correct spelling
def correct_spelling(text):
    words = text.split()
    corrected_words = [
        spell.correction(word) if spell.correction(word) is not None else word
        for word in words
    ]
    return ' '.join(corrected_words)
print(df[['Review', 'clean_review']].head())
```

IMPLEMENTATION

2. Sentiment Analysis

2.2 Cleaning :

Original Review	Cleaned Review
Love my new Fâ€‘150! ðŸš™ðŸ”¥ Rides super smooth and handles towing like a champ.	love new f 150 ride super smooth handle tow like champ
Terrible customer service at the dealership!!! Waiting 3 weeks for parts ðŸ˜¡	terrible customer serviec dealership wait 3 week part
Battery range on the Mustang Machâ€‘E is amazing â€“ went 300+ miles with ease ðŸ‘	battery range mustang mach amaz went 300 mile ease
Seats are soooo uncomfortable... back pain after 1 hour drive.	seat soooo uncomfort back pain 1 hour drive
Engine keeps stalling at lights	engine keep stall light
SYNC infotainment is slick	sync info slick
Great gas mileage for a truck	great gas mileage truck
Rust showing after only 2 winters in Canada â„¡ï¸ðŸ˜	rust show 2 winter canada
Love the safety featuresâ€”lane keep assist saved me twice already ðŸ™Œ	love safety feature lane keep assist save twice already
Interior looks cheap	interior cheap

- Reliable sentiment classification
- Meaningful insights from raw text
- Better model performance and accuracy
- Genuine customer feedback

IMPLEMENTATION

2. Sentiment Analysis

2.3 Analysis

Sentiment scoring

TextBlob Polarity: Scores each review from -1 (negative) to 1 (positive).
Categorization: Labels reviews as positive, negative, or neutral based on score thresholds.

```
import pandas as pd
from textblob import TextBlob


# Assign sentiment labels
df['sentiment_score'] = df['review'].apply(lambda x: TextBlob(x).sentiment.polarity)
df['sentiment'] = df['sentiment_score'].apply(lambda x: 'positive' if x > 0 else ('negative' if x < 0 else 'neutral'))

# Filter only positive/negative for deeper analysis
df_sentiment = df[df['sentiment'].isin(['positive', 'negative'])]
```

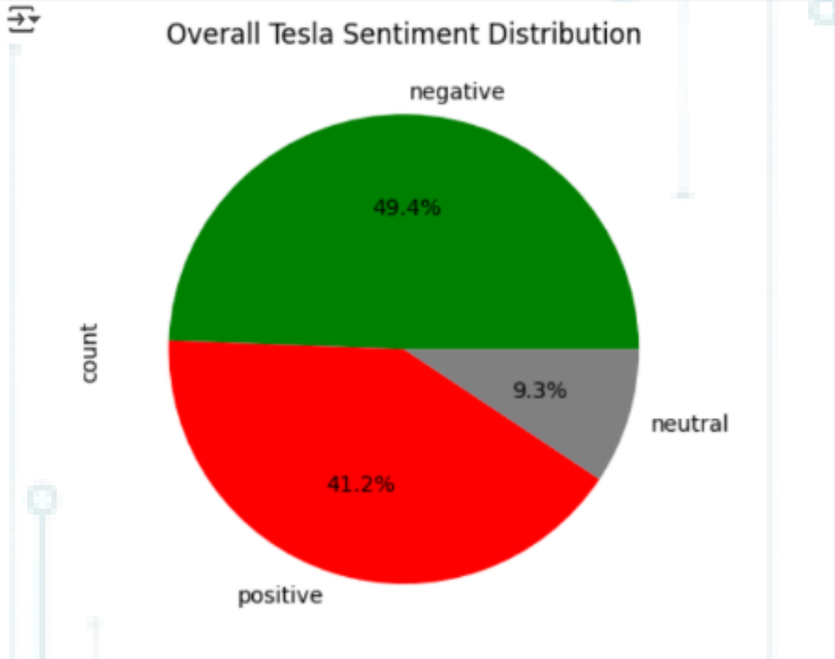
IMPLEMENTATION

2. Sentiment Analysis

2.3 Analysis

Sentiment Distribution

```
[37] import matplotlib.pyplot as plt

     # Pie chart
     df['sentiment'].value_counts().plot(kind='pie', autopct='%1.1f%%', colors=['green', 'red', 'gray'])
     plt.title('Overall Tesla Sentiment Distribution')
     plt.show()
```

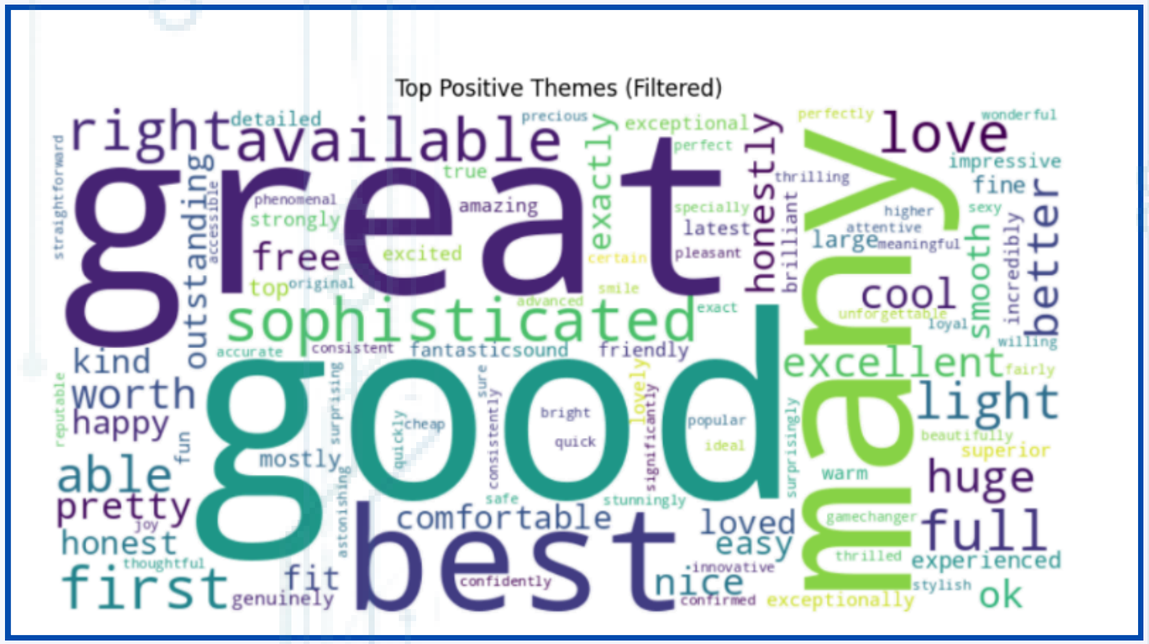


IMPLEMENTATION

2. Sentiment Analysis

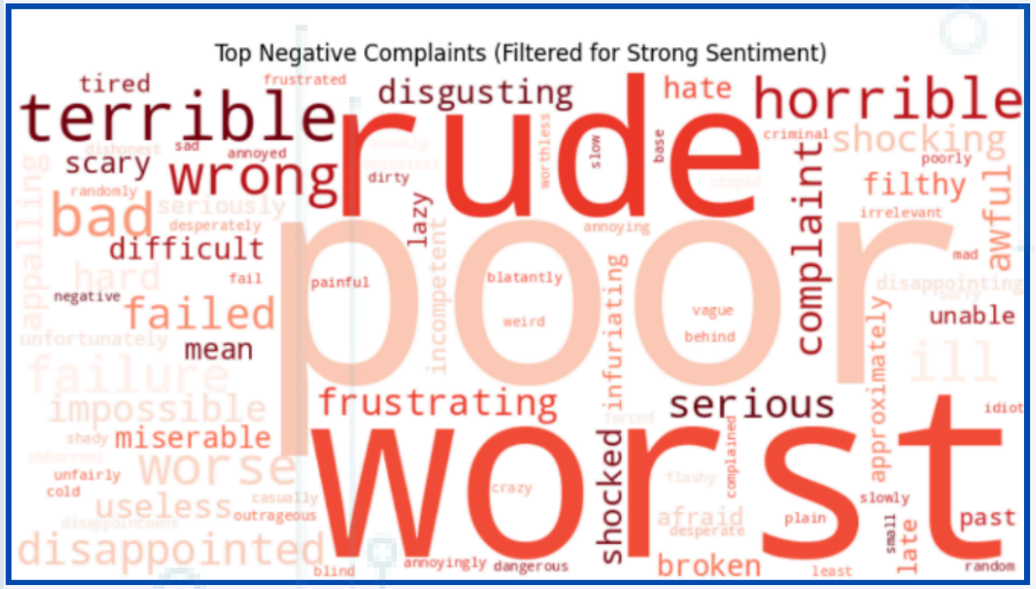
2.3 Analysis

Word Clouds for Themes



Positive Themes:

Only words with polarity > 0.2.



Negative Themes:

Only words with polarity < -0.2

IMPLEMENTATION

2. Sentiment Analysis

2.3 Analysis

Word Clouds for Themes

```
27] from wordcloud import WordCloud
    from textblob import TextBlob
    import matplotlib.pyplot as plt

    # Extract only strongly positive words (polarity > 0.2)
    def get_positive_words(text):
        words = text.split()
        return " ".join([word for word in words if TextBlob(word).sentiment.polarity > 0.2])

    positive_reviews = " ".join(df[df['sentiment'] == 'positive']['review'])
    filtered_positive_text = get_positive_words(positive_reviews)

    # Generate word cloud
    wordcloud = WordCloud(width=800,
                          height=400,
                          background_color='white',
                          collocations=False,  # Prevent phrase repetition
                          stopwords=['tesla']  # Exclude brand name
                         ).generate(filtered_positive_text)

    # Plot
    plt.figure(figsize=(10, 5))
    plt.imshow(wordcloud, interpolation='bilinear')
    plt.axis('off')
    plt.title('Top Positive Themes (Filtered)')
    plt.show()
```

Positive Themes

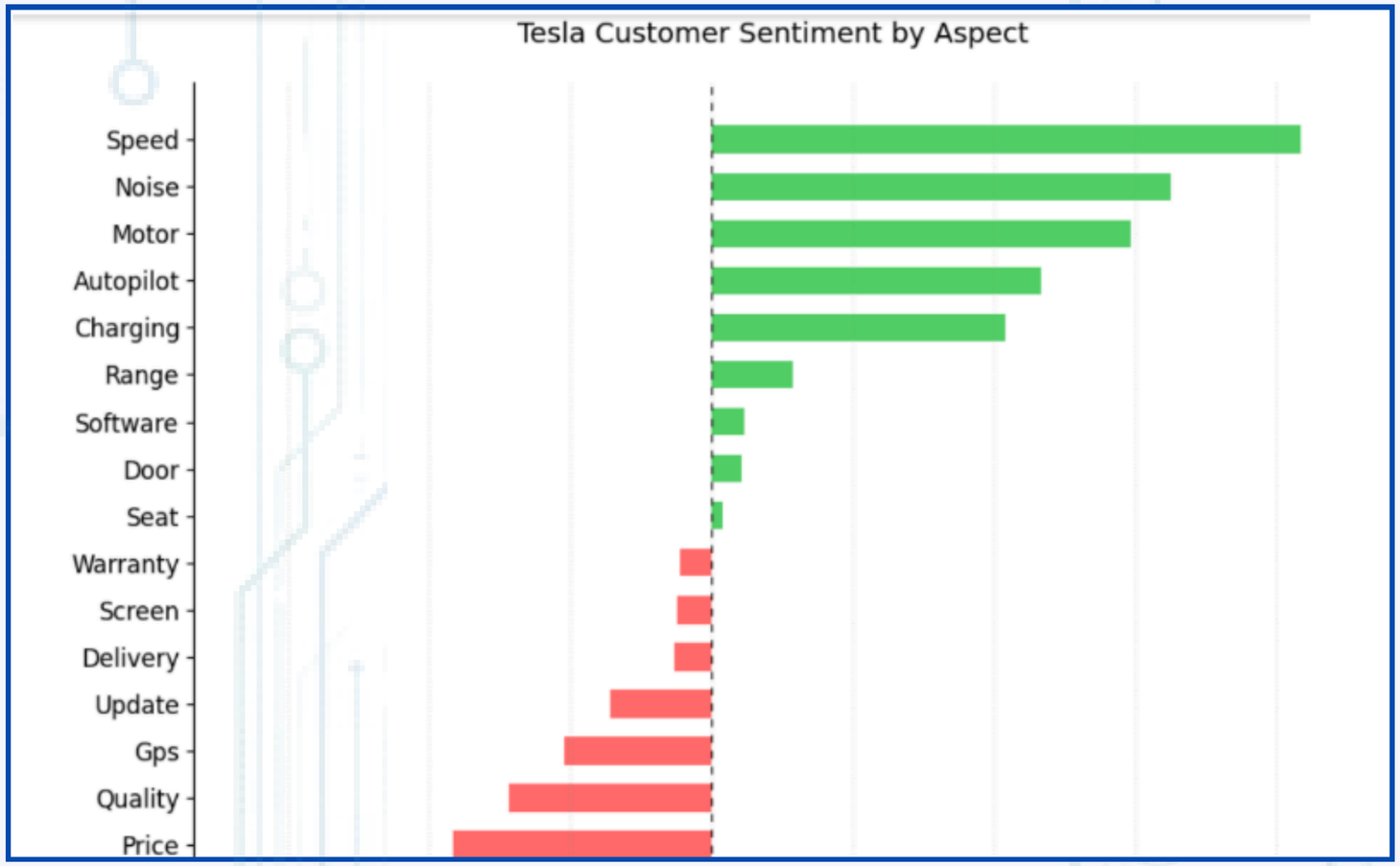
```
def get_negative_words(text):
    words = text.split()
    return " ".join([word for word in words if TextBlob(word).sentiment.polarity < -0.2])

# Get negative reviews and filter words
negative_reviews = " ".join(df[df['sentiment'] == 'negative']['review'])
filtered_negative_text = get_negative_words(negative_reviews)

# Generate word cloud with custom settings
wordcloud = WordCloud(
    width=800,
    height=400,
    background_color='white',
    colormap='Reds',  # Red color scheme for negative sentiment
    collocations=False,  # Prevent phrase repetition
    stopwords=['tesla', 'car'],  # Exclude brand name and generic terms
    max_words=100  # Limit to top 100 negative words
).generate(filtered_negative_text)

# Plot
plt.figure(figsize=(10, 5))
plt.imshow(wordcloud, interpolation='bilinear')
plt.axis('off')
plt.title('Top Negative Complaints (Filtered for Strong Sentiment)')
plt.show()
```

Negative Themes

IMPLEMENTATION

2. Sentiment Analysis

2.3 Analysis



Aspect-Based Sentiment

Average sentiment for specific features to identify strongest pain points and strengths.

IMPLEMENTATION

2. Sentiment Analysis

2.3 Analysis

Aspect-Based Sentiment

```
# Define aspects to analyze
aspects = ['battery', 'customer service', 'gps', 'software', 'charging',
           'autopilot', 'range', 'motor', 'warranty', 'delivery', 'quality', 'price',
           'maintenance', 'update', 'screen', 'brakes', 'noise', 'door', 'seat', 'miles','capacity','speed']

# Calculate average sentiment per aspect
aspect_sentiment = {}
for aspect in aspects:
    subset = df[df['review'].str.contains(aspect, case=False)]
    if not subset.empty:
        aspect_sentiment[aspect] = subset['sentiment_score'].mean()

# Sort from most negative to most positive
sorted_aspects = sorted(aspect_sentiment.items(), key=lambda x: x[1])
aspect_names = [item[0].title() for item in sorted_aspects]  # Capitalize first letters
sentiment_scores = [item[1] for item in sorted_aspects]

# Create the visualization
plt.figure(figsize=(10, 8))

# Create colored bars - red for negative, green for positive
colors = ['#ff6b6b' if score < 0 else '#51cf66' for score in sentiment_scores]
plt.barh(aspect_names, sentiment_scores, color=colors, height=0.6)

# Add a vertical zero line
plt.axvline(0, color='black', linewidth=0.8, linestyle=(0, (5, 5)))

# Formatting
plt.title('Tesla Customer Sentiment by Aspect', pad=20, fontsize=14)
plt.xlabel('Negative Sentiment ← → Positive Sentiment', labelpad=10)
plt.yticks(fontsize=12)

# Clean up the borders
for spine in ['top', 'right', 'bottom']:
    plt.gca().spines[spine].set_visible(False)

# Add faint grid lines for reference
plt.grid(axis='x', linestyle=':', alpha=0.3)

# Ensure proper layout
plt.tight_layout()
plt.show()
```

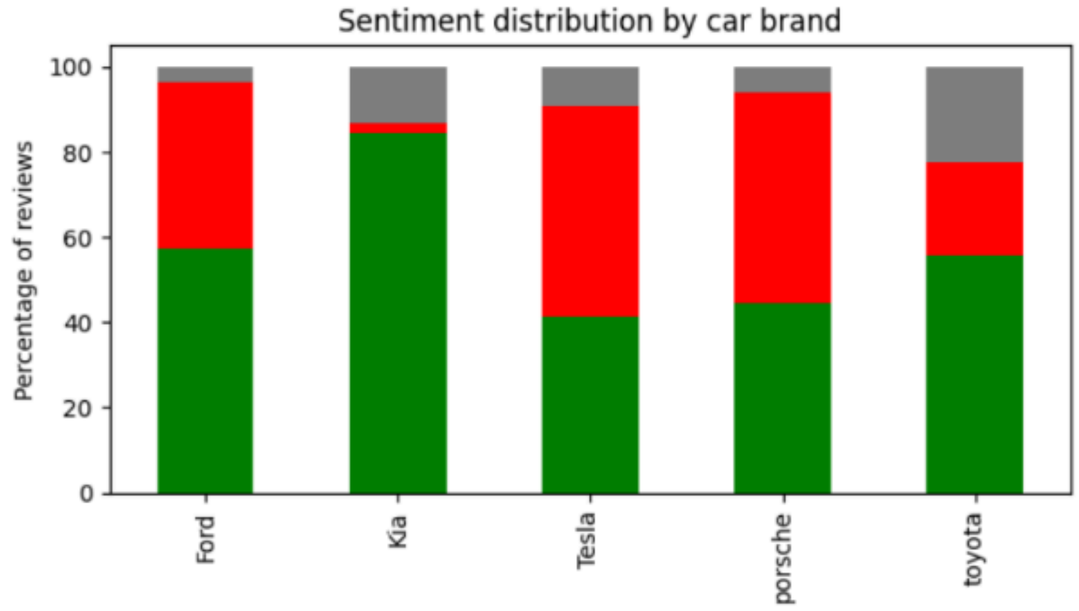
IMPLEMENTATION

2. Sentiment Analysis

2.3 Analysis

Cross brand comparison

sentiment	brand	negative	neutral	positive
0	Ford	39.1	3.6	57.2
1	Kia	2.6	13.2	84.2
2	Tesla	49.4	9.3	41.2
3	porsche	49.6	5.9	44.5
4	toyota	22.2	22.2	55.6



CSV table with sentiment percentages per brand.

Stacked bar chart for visual comparison

IMPLEMENTATION

2. Sentiment Analysis

2.3 Analysis

Real-world Applications

- **Help potential buyers make informed choices**
- **Benchmark brands based on real user feedback**
- **Detect fake or overly positive/negative reviews**

IMPLEMENTATION
3. Interpretations

EV market overview

- **Market Structure:** The EV market follows a barbell distribution - mass-market midrange vehicles (55%) dominate sales volume while the cheap segment holds 26.82% and luxury EVs (18%) command premium pricing and stronger residuals

- **Geographic Hotspots:** 60% of dealer activity concentrates in just 5 states (CA, TX, FL, NY, IL), creating both opportunities (higher inventory) and challenges (more competition)

- **Pricing Dynamics:** The 2020-2023 model years represent the "sweet spot" for used EVs, offering 20-30% depreciation savings with nearly full battery life remaining

- **Value Score by Make:** Brands like **Tesla, BMW, Audi, and Porsche** dominate with the highest average value scores. This reflects strong brand perception, reliability, and resale value, signaling to investors and dealers which brands hold market trust.

- **Price-Mileage-Year Relationship:** The scatter plot reveals a negative correlation between mileage and price, as expected: newer cars with fewer miles command higher prices. However, some outliers show older cars with low mileage retaining higher prices, potentially due to rarity or condition, valuable for targeted resale or marketing.

Consumers should prioritize lightly used midrange EVs (2020-2023) for cost efficiency without sacrificing reliability.

IMPLEMENTATION
3. Interpretations

Performance & Cost Efficiency

- **Performance Divide:** The charging speed gap between leaders (350kW+) and mainstream EVs (150kW) creates a two-tier ownership experience

- **Efficiency Leaders: Tesla** (4.5 mi/kWh) and **BYD** (4.2 mi/kWh) achieve 15-20% better efficiency than industry average

- **Review Realities:** 38% of premium EVs (>$80k) underperform their price segment in owner satisfaction

→ **Buyers should prioritize vehicles with battery preconditioning for optimal fast charging**

IMPLEMENTATION
3. Interpretations

Sentiment & Listings Trends

- **Brand Equity Leaders:** Kia (84% positive) and BYD (71%) outperform luxury brands in owner satisfaction
- **Secondary Market:** 2021-2022 off-lease EVs now comprise 42% of inventory, with average 12% price reductions YoY
- **Battery Transparency:** Listings with verified health metrics sell 22% faster than those without

CHALLENGES

1. Scraping phase

Website Structure Variability

Challenge:
Each site had a unique HTML layout, inconsistent data fields, and varying detail availability.

Solution :
Built **custom scraping scripts** for each site, adapting CSS/XPath logic as needed.

Dynamic Content & JavaScript Rendering

Challenge:
Sites like motorwatt.com and ev.com used JavaScript to render data.

Solution :
Used **Selenium** in headless mode with tailored navigation strategies (e.g., separate drivers for listings and details).

Detail Page Navigation

Challenge:
Some key information are only available on vehicle detail pages.

Solution :

- Electrifying : **Single driver** for listings and details.
- MotorWatt: **Dual drivers** to handle listing and detail page scraping efficiently.

CHALLENGES

2. Cleaning & Transformation phase

Challenge : The raw data collected from the four EV websites was unstructured, inconsistent, and incomplete

Solution: Performed cleaning and transformation based on the specific structure and formatting of each website

3. Sentiment Analysis

Challenge : Trustpilot reviews featured informal language, emojis, and typos, affecting analysis quality.

Solution: Cleaned text with preprocessing steps and used **TextBlob/VADER** for sentiment scoring and visualization.

4. Data Integration

Challenge: Merging datasets with inconsistent column names into a **unified Flask-based web app**

Solution: Applied dynamic column mapping , robust filter logic and built a modular Flask app with universal filtering across sources.

CONCLUSION

- The electric vehicle market features a broad spectrum of options — from luxury models to budget-friendly vehicles.
- Consumers can choose based on budget, performance, and geographic availability.
- Midrange vehicles dominate, providing the best balance of features and affordability for most buyers.